

Report

Describe your approach:

How does your game-playing program select actions throughout the game?

Throughout this assignment, we developed several different agents, ranging from the easiest random agent to a greedy agent with a heuristic function and lastly an agent that utilised minimax with alpha-beta pruning.

Greedy

With our greedy agent, we initially had only one goal for our evaluation function: No. of red pieces - blue pieces. Or vice versa. As we continued to observe the games being played out, however, we then turned towards also rewarding qualities such as occupying central blocks or having a good spread. (These bonuses were weighted according to their importance.)

Additionally, we realised that since the game grid spans 11x11, to have more pieces than your opponent, the optimal amount of pieces to have on a single line are 5 or 6. Beyond that, you take the risk of your opponent being able to clear your row/column. In such a case, you would lose 7+ blocks while the opponent would only lose 1 - 4 blocks in clearing your row/column.

Furthermore, as we played out more games, we also realised that often even though our agent had material advantage in the game, it was often up to 'luck' who would run out of legal moves first (looking only one step ahead). To avoid this we utilised a mobility function, and rewarded the game states that could maintain a certain amount of mobility points (in hopes that this would keep it surviving for longer).

To make our 'mobility' function efficient, we decided to prune the search. Instead of generating all possible legal moves a given state could produce, we decided to perform early stoppage, breaking the loop everytime 1 legal move was found, from a certain coordinate, in a certain direction. In doing so, we were able to significantly reduce the complexity of computing mobility.

Minimax

Our minimax algorithm works by taking in the current *board* state, the desired *depth*, the current known *alpha* and *beta*, and a boolean value, with a false value implying ‘minimise’ and a true value, ‘maximise’.

Our minimax algorithm would recursively call itself, until a terminal state was reached, or a cut-off depth ($=0$). From these terminal nodes, it would then use our evaluation function ‘compute_score’ to calculate the utility of the state. Returning this value.

Eventually, these utility values would be passed up the minimax tree, with pruning taking place wherever possible, and a final ‘**best move**’ would be returned to the calling def(action) function.

Performance evaluation:

How effective is your game-playing program?

To evaluate our program’s effectiveness, we utilised the script shared on the ed discussion forum that helped to simulate 100 games being played. Despite that, however, we still found it to execute too slowly, as it would still only execute game by game, so it was not the most suitable for testing volume. In the future, we would hope to find a more efficient method to run these trials in bulk, to help us tune our various hyperparameters more finely.

and, if so, how have you selected which is the most effective?

The agent we finally decided on was our greedy agent that evaluated game states based on points, spread, and mobility. As this game of tetress had a very high branching factor, especially as the game progressed, we found it difficult with the minimax algorithm to reach a decent depth, because of its exponential time complexity.

Besides that, we found little value in constantly trying to optimise the algorithm simply to increase depth by 1. It did not improve performance much, as the game of tetris itself is highly volatile, due to the 'line clearing' rule. Furthermore, as the complexity of our evaluation function increased, it became increasingly hard for depth to increase along with it. As with a greater depth, there were exponentially more potential states to evaluate.

Lastly, we also found it challenging to generate a good move ordering, as it is not a simple task to determine whether a Z shaped tetrimino is better than an L shaped one. Considering all these reasons, therefore, we decided to use a greedy algorithm with a better evaluation function rather than a minimax algorithm with low depth and a simple evaluation function.

Other aspects:

Are there any other important creative or technical aspects of your work?

In order to improve efficiency, such that our minimax algorithm could achieve greater depths, we attempted several optimisations.

1. def precompute_moves_for_coord(r, c)

Using this function, we managed to optimise our '*tetriminos move generation*'. We computed a dictionary of possible moves for every coordinate, with the keys of the dictionary being unique coordinates on the board, and the values being another dictionary holding all possible shapes in each direction.

This greatly improved our efficiency, halving our runtime, however, it was still not enough as the function `is_legal_placement(self, board, current_shape)`: still suffered from the exponential time complexity, and due to the nature of the problem, calculating whether a move is legal is dependent on the current board and runtime and thus cannot be effectively pre-computed.

2. Pruning move generation

Another method we used to improve the efficiency of our move generation was pruning the search whenever the adjacent coordinate was '*not None*' (occupied). This helped in reducing our search tree as checking those adjacent coordinates meant we no longer needed to attempt fitting any shapes in that direction if it was found to be occupied.

3. Simulating and Reverting vs copying

While trying to optimise our algorithm further we found that actions involving copying a dictionary, '*the game state*', was computationally expensive both time and memory-wise. In the case of this game, a fully occupied board would mean the dictionary potentially occupies 121 entries.

Copying such a dictionary therefore would take $O(n)$ operations, where $n = 121$. While this seems trivial, this operation was needed within the minimax algorithm and recursively called for every potential state found.

Therefore, we decided to optimise this by:

1. Simulating the move being placed on the board
2. Simulate any lines that may be removed as a result
3. Pass this simulated board into the recursive function's parameter
4. Undo any lines previously cleared
5. Undo the move previously done

Supporting work

Have you completed any other work to assist you in the process of developing your game-playing program?

To investigate why our algorithm was performing slower than expected, we used cProfile. Our results appeared this way:

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1803/1	0.043	0.000	25.779	25.779	/workspaces/part_b/agent_minimax/program.py:212(minimax)
331	0.987	0.003	15.655	0.047	/workspaces/part_b/agent_minimax/program.py:107(generate_legal_moves)
380890	1.672	0.000	13.644	0.000	/workspaces/part_b/agent_minimax/program.py:95(is_legal_placement)
1252689	4.451	0.000	9.321	0.000	{method 'get' of 'dict' objects}
2832147	4.575	0.000	7.084	0.000	<string>:2(__hash__)
384494	2.772	0.000	6.580	0.000	/workspaces/part_b/referee/game/actions.py:20(coords)
1472	1.888	0.001	6.551	0.004	/workspaces/part_b/agent_minimax/program.py:28(compute_score)
1802	0.872	0.000	3.314	0.002	/workspaces/part_b/agent_minimax/program.py:153(clear_full_rows_and_columns)
2832147	2.509	0.000	2.509	0.000	{built-in method builtins.hash}
1287915	1.747	0.000	1.747	0.000	<string>:2(__eq__)
47756	0.189	0.000	0.519	0.000	/workspaces/part_b/referee/game/coord.py:114(__add__)
515373	0.519	0.000	0.519	0.000	

From, statistics like this, therefore, we were able to discover which function calls were taking the longest, narrowing down our options on what we needed to optimise first.